

Bitset Algorithms for the Generative Structure of Chemical Organizations: From Elementary Reaction Closures to All Persistent Modules

Tomás Veloz^{1,*}
Paolo Bassi²

¹Templeton World Charity Foundation Postdoctoral Fellow, CLEA, Vrije Universiteit Brussel, Belgium

²...

*tveloz@gmail.com

Abstract

Motivation: Chemical Organization Theory (COT) identifies the closed, self-maintaining sub-networks of a reaction network – *persistent modules* – that constrain its long-run dynamical behaviour. Deciding whether a network contains one is NP-hard, and the established bottom-up algorithms enumerate the lattice of closed sets, most of which are generatively irrelevant, so exact computation has been confined to networks of a few dozen reactions while genome-scale metabolic reconstructions reach thousands. A companion theoretical study shows that every persistent module is generated from *elementary reaction closures* (ERCs) by two irreducible relations, fundamental synergy and fundamental complementarity, but stops at an existence theorem.

Results: We give and implement an algorithm that computes, exactly, all elementary persistent modules (EPMs) and all elementary-supported persistent modules (ESPMs) of a reaction network from its ERCs. The search is carried out entirely in ERC-index space: each partial module is three incrementally updated bitmasks (species present, species produced, species still externally required), so no reaction scan or closure recomputation is ever needed after the ERC hierarchy, the fundamental-synergy hypergraph and the fundamental-complementarity graph have been built once. A module grows by three moves – attaching a minimal producer of an open requirement, attaching a synergy partner, or, once the module is already self-maintaining, attaching a minimal consumer of a species it already produces or lifting to a hierarchy ancestor of one of its constituents – and the algorithm deliberately searches only this *generated core*: unions of already-persistent modules that share no synergy or complementarity (*latent joins*) are excluded from the search and reconstructed in a single closure computation on demand. We validate correctness by exhaustive comparison against a brute-force oracle on hand-built networks, a fully worked network exhibiting every mechanism including a sterile ERC, and a sample of real BioModels networks up to the size at which brute force remains tractable, finding exact agreement throughout. On a genome-scale metabolic reconstruction (825 species, 1296 reactions, 712 ERCs) the algorithm finds 72 EPMs and 1124 ESPMs across 17 orders in under 20 seconds on one desktop core – a scale at which brute-force enumeration is entirely infeasible.

Availability: Implemented in the open-source `cot_gen` library; <https://github.com/tveloz/pyCOT>.

Contact: tveloz@gmail.com

Keywords: Chemical Organization Theory, reaction networks, elementary reaction closures, synergy, complementarity, persistent modules, bitset algorithms.

1 Introduction

Chemical Organization Theory (COT) studies the long-run behaviour of a reaction network by identifying its closed, self-maintaining subsets, or *organizations* [2]. A companion theoretical study [1] isolates the minimal building blocks from which every such subset is assembled – *elementary reaction closures* (ERCs) – organises them into a containment hierarchy, and identifies two relations between ERCs, *synergy* (a combination fires a reaction neither part fires alone) and *complementarity* (one part produces a species the other requires), whose most restrictive, *fundamental* forms are proven sufficient to generate every persistent module. That study proves existence but gives neither an algorithm nor an implementation: it stops at brute-force statistics over 438 biological networks.

This paper supplies the missing computational layer. We describe, and have implemented and validated, the `cot_gen` library, which turns the companion study’s generative theorems into a concrete pipeline: it discovers ERCs and their hierarchy, computes basic/maximal/fundamental synergies and basic/pure/fundamental complementarities, identifies the primitive ERCs and their synergy-generated reach, and – the central contribution of this paper – computes *all* elementary persistent modules (EPMs) and elementary-supported persistent modules (ESPMs) of a reaction network, of any order, by a bounded-branching search in ERC-index space rather than by enumerating the lattice of closed sets. Every stage is cross-validated against an independent brute-force oracle, on hand-built networks, on a fully worked example, and on a sample of real BioModels networks up to the size at which the oracle itself remains tractable.

Structure. Section 2 reviews the COT definitions this paper builds on. Sections 3–7 describe ERC discovery, the containment hierarchy, synergy, complementarity, and the primitive-ERC generative basis. Section 8 gives the central contribution: the ERC-index-space representation and the two-mode traversal that computes all EPMs and ESPMs while searching only the generated core. Section 9 describes the bitset implementation. Section 10 describes the oracle-first validation protocol and its results, including a fully worked example. Section 11 reports a representative genome-scale run. Section 12 concludes and lists the algorithmic extensions this implementation makes possible.

2 Background

2.1 Reaction Networks and the E_0 Quotient

A *reaction network* (RN) is a tuple $(M, \mathcal{R})$ where $M = \{s_1, \dots, s_n\}$ is a finite set of *species* and $\mathcal{R} = \{r_1, \dots, r_m\}$ is a finite set of *reactions*. Each reaction r has a *support* $\text{supp}(r) \subseteq M$ (species consumed) and a *product set* $\text{prod}(r) \subseteq M$ (species produced).

Definition 1 (E_0 set). *The inflow set $E_0 \subseteq M$ is the closure of all species produced by reactions with empty support (inflow reactions): $E_0 = \text{cl}(\bigcup_{r: \text{supp}(r)=\emptyset} \text{prod}(r))$.*

All subsequent computation uses the *quotiented* support and products, $\text{supp}_q(r) = \text{supp}(r) \setminus E_0$ and $\text{prod}_q(r) = \text{prod}(r) \setminus E_0$; reactions with $\text{supp}_q(r) = \emptyset$ play no further role in ERC computation, since E_0 already sits inside every persistent module and is suppressed below.

Definition 2 (Closure). *The closure $\text{cl}(X)$ of a species set $X \subseteq M$ is the smallest $Y \supseteq X$ such that $\text{supp}_q(r) \subseteq Y \Rightarrow \text{prod}_q(r) \subseteq Y$ for all r .*

2.2 Elementary Reaction Closures

Definition 3 (ERC [1]). For each reaction r with $\text{supp}_q(r) \neq \emptyset$, its ERC mask is $E_r = \text{cl}(\text{supp}_q(r))$. The ERC set is $\mathcal{E} = \{E_r : r \in \mathcal{R}, \text{supp}_q(r) \neq \emptyset\}$. Distinct reactions with the same closure are grouped: $\mathcal{E}$ is the set of distinct closures.

Definition 4 (Requirement and production masks). For $E \in \mathcal{E}$ let $\mathcal{R}_E = \{r : \text{supp}_q(r) \subseteq E\}$ be the reactions active within E . Then

$$\text{prod}(E) = \bigcup_{r \in \mathcal{R}_E} \text{prod}_q(r), \quad \text{req}(E) = \left(\bigcup_{r \in \mathcal{R}_E} \text{supp}_q(r) \right) \setminus \text{prod}(E).$$

E is persistent (a P-ERC) iff $\text{req}(E) = \emptyset$.

ERCs are partially ordered by inclusion; the Hasse diagram of direct containments is a forest [1].

2.3 Semi-Organizations and Persistent Modules

Definition 5 (Semi-organization (SO) [1]). A species set $X \subseteq M$ is a semi-organization iff it is (i) closed: $\text{prod}(\mathcal{R}_X) \subseteq X$; (ii) semi-self-maintaining (SSM): $\text{req}(X) = \emptyset$; (iii) reactive: $\mathcal{R}_X \neq \emptyset$; (iv) connected: two species of X are connected iff they co-occur in the support or product of some active reaction, and this relation spans all of X . We write SO for the set of all semi-organizations of a network, our computational target.

Definition 6 (EPM and ESPM order [1]). $X \in SO$ is an elementary persistent module (EPM) if its only proper sub-SO is $E_\emptyset$ (the empty set, once $E_\emptyset$ is quotiented out); write $\text{ord}(X) = 0$. Otherwise X is an elementary-supported persistent module (ESPM) of order $\text{ord}(X) = 1 + \max\{\text{ord}(Y) : Y \in SO, \emptyset \subsetneq Y \subsetneq X\}$.

2.4 Synergy and Complementarity

Every reactive closed set is a join of ERCs, so ERCs generate SO [1]. Two ERCs are *synergetic* if their join fires a reaction neither fires alone; and *complementary* if one supplies a requirement of the other, measured by $\text{supl}(E \rightarrow E') = \text{prod}(E) \cap \text{req}(E')$. The *fundamental* forms of each relation minimise the participants (a fundamental synergy uses the smallest reactant ERCs producing the largest target; a fundamental complementarity carries a species from a $\subseteq$ -minimal producer to a $\subseteq$ -minimal consumer – exact definitions in Sections 5 and 6). The companion study proves that every $X \in SO$ admits a *fundamental generator*: an irreducible sequence of ERC attachments from $E_\emptyset$, each of which is either a fundamental synergy or a fundamental complementarity, and that this suffices to reach every persistent module without ever forming a reaction-inert combination of unrelated closed sets [1]. Sections 3 through 7 give the bitset algorithms that compute the ERC hierarchy and these fundamental relations; Section 8 gives the algorithm that walks them to enumerate SO itself.

3 ERC Discovery

Closure is computed by a Horn/Dowling–Gallier-style propagation [3]: a reactant counter per reaction is decremented as species are added, and a reaction fires the instant its counter reaches zero, avoiding the naive fixpoint’s repeated rescans. Each $\text{MinBas}(E)$ is the $\subseteq$ -minimal antichain of reaction supports whose closure is E ; a species set ignites E iff it contains some element of $\text{MinBas}(E)$ (Lemma 1 below), which is what makes bases the right domain for detecting synergy in Section 5.

Algorithm 1 ComputeERCs($\mathcal{R}, E_0$)

```
1: groups  $\leftarrow \{\}$   $\triangleright$  mask  $\mapsto$  list of reaction indices
2: for each  $r \in \mathcal{R}$  with  $\text{supp}_q(r) \neq \emptyset$  do
3:    $E \leftarrow \text{cl}(\text{supp}_q(r))$ 
4:   groups[ $E$ ].append( $r$ )
5: end for
6: ercs  $\leftarrow []$ 
7: for each  $(E, \text{rxns})$  in groups do
8:   bases  $\leftarrow \text{MinBas}(\{\text{supp}_q(r) : r \in \text{rxns}\})$ 
9:   Compute req( $E$ ), prod( $E$ ) from  $\mathcal{R}_E$ 
10:  ercs.append(ERCData( $E, \text{rxns}, \text{bases}, \text{req}(E), \text{prod}(E)$ ))
11: end for
12: return ercs sorted by  $|E|$ 
```

Complexity. Closure via Horn propagation costs $O(\sum_r |\text{supp}_q(r)|)$ amortised per call; $O(|\mathcal{R}|)$ closure calls give $O(|\mathcal{R}| \cdot \sum_r |\text{supp}_q(r)|)$ total. The MinBas antichain filter costs $O(k^2)$ per group of k same-closure reactions.

4 Containment Hierarchy

Definition 7 (ERC order). $E_i \leq E_j \iff E_i \subseteq E_j$ as species bitmasks.

The Hasse diagram stores only direct-cover edges (no intermediate ERC between them); the algorithm first computes all containment pairs via the bitset predicate $(m_i \& m_j) = m_i$, then removes transitively implied edges, and finally derives, for every ERC, its full ancestor and descendant sets by transitive closure – these are the sets the traversal of Section 8 queries directly (**parents**, **ancestors**, **descendants**).

Complexity. $O(|\mathcal{E}|^2)$ for the pairwise containment scan and Hasse reduction; $O(|\mathcal{E}|^2)$ for the ancestor/descendant closure.

5 Synergy

5.1 Definitions [1]

Definition 8 (Basic synergy). $(E_i, E_j) \rightsquigarrow E_k$ is a basic synergy iff E_i, E_j, E_k are pairwise incomparable and $\exists b \in \text{MinBas}(E_k)$ with $b \subseteq (E_i \cup E_j)$, $b \not\subseteq E_i$, $b \not\subseteq E_j$: the joint species set ignites a basis of E_k that neither part ignites alone.

Definition 9 (Maximal synergy). $(E_i, E_j) \rightsquigarrow^{\max} E_k$ is maximal iff it is basic and no $E_{k'} \supsetneq E_k$ is also a basic synergy target of the pair (E_i, E_j) .

Definition 10 (Fundamental synergy). A maximal synergy $(E_i, E_j) \rightsquigarrow^{\max} E_k$ is fundamental (written $E_i + E_j \twoheadrightarrow E_k$) iff both reactant slots are irreducible: no $E_{i'} \subsetneq E_i$ gives $(E_{i'}, E_j) \rightsquigarrow^{\max} E_k$, and no $E_{j'} \subsetneq E_j$ gives $(E_i, E_{j'}) \rightsquigarrow^{\max} E_k$.

Algorithm 2 ComputeFundamentalSynergies-PairFirst($\mathcal{E}$, hier)

```
1: for all incomparable pairs  $(i, j)$  do
2:   for each  $k$  not a descendant of  $i$  or  $j$  do
3:     if  $\exists b \in \text{MinBas}(k) : b \subseteq m_i | m_j, b \not\subseteq m_i, b \not\subseteq m_j$  then
4:        $\text{basic}[i, j].\text{add}(k)$ 
5:     end if
6:   end for
7: end for
8:  $\text{maximal} \leftarrow \{(i, j, k) \in \text{basic} : \nexists k' \text{ with } m_k \subsetneq m_{k'} \in \text{basic}[i, j]\}$ 
9: for each  $(i, j, k) \in \text{maximal}$  do
10:  if  $\nexists i' : m_{i'} \subsetneq m_i \wedge (i', j, k) \in \text{maximal}$  and  $\nexists j' : m_{j'} \subsetneq m_j \wedge (i, j', k) \in \text{maximal}$  then
11:     $\text{emit fundamental}(i, j, k)$ 
12:  end if
13: end for
```

Algorithm 3 ComputeFundamentalSynergies-BasisFirst($\mathcal{E}$, hier)

```
1: for each target  $k \in \mathcal{E}$  do
2:   for each basis  $b \in \text{MinBas}(k)$  do
3:      $\text{relevant} \leftarrow \{i : \emptyset \subsetneq (b \& m_i) \subsetneq b\}$ 
4:     for each incomparable pair  $(i, j)$  in relevant with  $(b \& m_i) | (b \& m_j) = b$  do
5:        $\text{basic}[i, j].\text{add}(k)$ 
6:     end for
7:   end for
8: end for
9: apply the same maximal and per-slot fundamentality filters as Algorithm 2
```

5.2 Pair-First Algorithm

Complexity. $O(|\mathcal{E}|^3 \cdot B)$ where $B = \max_k |\text{MinBas}(E_k)|$; used as the reference implementation and for cross-validating the basis-first algorithm below.

5.3 Basis-First Algorithm (default)

The pair-first scan checks every incomparable pair against every target's bases, $O(|\mathcal{E}|^3 \cdot B)$ overall, even though on sparse networks most ERC pairs share no species with any given basis. Basis-first inverts the loop order: for each target k and each basis $b \in \text{MinBas}(k)$, only the ERCs that cover *some but not all* of b are relevant reactant candidates – an ERC covering all of b ignites k by itself and is not a synergy source, and one covering none of b contributes nothing to it.

Proposition 1 (Equivalence). *Algorithms 2 and 3 report the same basic synergy set. For $(i, j) \rightarrow k$: let $b_i = b \& m_i$, $b_j = b \& m_j$. $(i, j) \rightsquigarrow k$ via b in the pair-first sense iff $b_i \neq \emptyset$, $b_j \neq \emptyset$, $b_i \neq b$, $b_j \neq b$, and $b_i | b_j = b$ – exactly the basis-first membership test, by De Morgan's laws on the joint-cover predicate.*

The maximal and fundamental filters are reused unchanged from Algorithm 2. Basis-first is the algorithm used by default throughout this paper and by `cot_gen`'s pipeline script; the pair-first algorithm is retained purely as an independent cross-check (Section 10).

Algorithm 4 ComputeBasicComplementarity($\mathcal{E}$, hier)

```
1: for all incomparable pairs  $(i, j)$  do
2:   fwd  $\leftarrow p_i \ \& \ r_j$ ;   bwd  $\leftarrow p_j \ \& \ r_i$ 
3:   if fwd  $\neq 0$  or bwd  $\neq 0$  then
4:     emit BasicComp( $i, j$ , fwd, bwd, is_pure)
5:   end if
6: end for
```

Complexity. Bounded by $O(|\mathcal{E}| \cdot B \cdot \max_b |\text{relevant}(b)|^2)$, which is markedly smaller than $O(|\mathcal{E}|^3 \cdot B)$ whenever bases are locally concentrated – the typical case for metabolic and signalling networks (Section 11).

6 Complementarity

6.1 Definitions [1]

Definition 11 (Supply). *The supply from E to E' is $\text{supl}(E \rightarrow E') = \text{prod}(E) \cap \text{req}(E')$. In bitsets: $\text{prod_mask}_E \ \& \ \text{req_mask}_{E'}$.*

Definition 12 (Basic complementarity). *(E, E') are basic complementary iff $\text{supl}(E \rightarrow E') \cup \text{supl}(E' \rightarrow E) \neq \emptyset$: at least one ERC produces a species the other requires. Only incomparable pairs are considered, since for $E \subsetneq E'$, $\text{prod}(E) \subseteq \text{prod}(E')$ gives $\text{supl}(E \rightarrow E') = \emptyset$ trivially.*

Definition 13 (Pure complementarity). *(E, E') are purely complementary iff they are basic complementary and not synergetic (no basic synergy involves the pair). This isolates requirement reduction attributable directly to ERC-pair supply from requirement reduction that is really an artefact of a jointly activated reaction – see Remark 1 below.*

Definition 14 (Fundamental complementarity). *For species $s \in \text{supl}(E \rightarrow E')$, the supply is fundamental iff $E \in \text{minprod}(s)$ and $E' \in \text{mincons}(s)$, where $\text{minprod}(s) = \min_{\subseteq} \{E \in \mathcal{E} : s \in \text{prod}(E)\}$ and $\text{mincons}(s) = \min_{\subseteq} \{E \in \mathcal{E} : s \in \text{req}(E)\}$ are the $\subseteq$ -minimal producers and consumers of s . We write $E \rightleftharpoons_s E'$.*

Remark 1 (Complementarity does not certify closed-set-level requirement reduction). Joining two persistent-module constituents can strictly reduce their combined requirement, $\text{req}(X \vee Y) \subsetneq \text{req}(X) \cup \text{req}(Y)$, purely because the join fires a new (synergetic) reaction whose product closes a requirement that neither part supplied on its own; the pair itself then carries *zero* basic complementarity under Definition 4, even though the module as a whole becomes more self-sufficient. This is precisely why pure complementarity and fundamental synergy are tracked as separate, non-overlapping relations rather than folded into a single “requirement shrank” predicate, and it is exercised directly by our test suite (Section 10).

6.2 Algorithms

Algorithm 5 builds two inverted indices over the emitted `FundComp` triples – `comp_by_species[s]`, the minimal producers of s , and `comp_consumers_by_species[s]`, the minimal consumers of s – because the traversal of Section 8 needs both directions: producers to close an open requirement, consumers to discharge productive novelty once a module is already self-maintaining.

Algorithm 5 ComputeFundamentalComplementarity($\mathcal{E}$)

```
1: for each species bit  $s$  do
2:    $\text{prod}_s, \text{req}_s \leftarrow$  producers, requirers of  $s$ 
3:   if  $\text{prod}_s = \emptyset$  or  $\text{req}_s = \emptyset$  then
4:     continue
5:   end if
6:    $\text{minprod}(s), \text{mincons}(s) \leftarrow \subseteq$ -minimal antichains of  $\text{prod}_s, \text{req}_s$ 
7:   for  $\pi \in \text{minprod}(s), \gamma \in \text{mincons}(s), \pi \neq \gamma$  do
8:     emit FundComp( $\pi, \gamma, s$ ) ▷ indexed by both producer  $\pi$  and consumer  $\gamma$ 
9:   end for
10: end for
```

Algorithm 6 ComputeGenerators($\mathcal{E}, \mathcal{F}$)

```
1:  $\mathcal{P} \leftarrow \{E \in \mathcal{E} : E \notin \{E_k : (i, j, k) \in \mathcal{F}\}\}$ 
2:  $\mathcal{B} \leftarrow \mathcal{P}$ 
3: while  $\mathcal{B}$  changes do
4:   for each  $(i, j, k) \in \mathcal{F}$  do
5:     if  $E_i, E_j \in \mathcal{B}$  then
6:        $\mathcal{B} \leftarrow \mathcal{B} \cup \{E_k\}$ 
7:     end if
8:   end for
9: end while
10: return ( $\mathcal{P}, \mathcal{B}, |\mathcal{B}|/|\mathcal{E}|$ )
```

Complexity. Algorithm 4: $O(|\mathcal{E}|^2)$. Algorithm 5: $O(|M| \cdot |\mathcal{E}|^2)$, dominated by the minprod / mincons antichain filters.

7 Primitive ERCs and the Generative Basis

Definition 15 (Primitive ERC). $E \in \mathcal{E}$ is primitive iff it is not the target of any fundamental synergy.

Definition 16 (Synergy reachability). The basis reach $\mathcal{B}$ is the smallest superset of the primitive ERCs $\mathcal{P}$ closed under fundamental synergy: if $E_i, E_j \in \mathcal{B}$ and $E_i + E_j \rightarrow E_k$, then $E_k \in \mathcal{B}$.

This synergy-only reachability closure identifies how much of the ERC poset is explained purely by combining primitives via fundamental synergy; it is a diagnostic on $\mathcal{E}$ (Section 11 reports coverage on a genome-scale network) and is distinct from, and computed before, the full module search of Section 8, which also draws on complementarity and hierarchy structure.

Complexity. $O(|\mathcal{E}| \cdot |\mathcal{F}|)$.

8 Computing All Persistent Modules

This section gives the paper’s central contribution: an algorithm that computes *every* EPM and ESPM of a reaction network – not just single-ERC or pairwise combinations – directly from the structures of Sections 3–7, without ever computing a reaction-level closure during the search itself.

Algorithm 7 ExtendState(S , sp, req, prod, j)

```
1: implied  $\leftarrow \{j\}$ ; sp'  $\leftarrow$  sp |  $m_j$ 
2: queue  $\leftarrow \{j\} \cup \text{descendants}(j)$ 
3: while queue nonempty do
4:   pop  $\ell$  from queue
5:   for each  $(a, b, k)$  with  $\ell \in \{a, b\}$ ,  $k \notin \text{implied}$  do
6:     let  $o$  be the other reactant slot ( $a$  or  $b$ )
7:     if  $m_o \& \text{sp}' = m_o$  then  $\triangleright$  species-subset ignition test
8:       implied  $\leftarrow$  implied  $\cup \{k\}$ 
9:       sp'  $\leftarrow$  sp' |  $m_k$ 
10:      push  $k$  and descendants( $k$ ) onto queue
11:    end if
12:  end for
13: end while
14: add_req, add_prod  $\leftarrow \bigcup_{i \in \text{implied}} \text{req}(E_i)$ ,  $\bigcup_{i \in \text{implied}} \text{prod}(E_i)$ 
15: prod'  $\leftarrow$  prod | add_prod
16: return ( $S \cup \text{implied}$ , sp', (req | add_req)  $\&$   $\sim$  prod', prod')
```

8.1 Modules as ERC-index-space states

Lemma 1 (Ignition). *For closed X and $E \in \mathcal{E}$, $E \subseteq X$ iff some $B \in \text{MinBas}(E)$ has $B \subseteq X$.*

Proof. ($\Rightarrow$) any $B \subseteq E \subseteq X$. ($\Leftarrow$) $B \subseteq X$ closed gives $E = \text{cl}(B) \subseteq X$. $\square$

A partial module under construction is represented not as a species set but as a set $S \subseteq \{1, \dots, |\mathcal{E}|\}$ of ERC indices, together with three incrementally maintained bitmasks:

$$\text{sp}(S) = \bigcup_{i \in S} E_i, \quad \text{prod}(S) = \bigcup_{i \in S} \text{prod}(E_i), \quad \text{req}(S) = \left(\bigcup_{i \in S} \text{req}(E_i) \right) \setminus \text{prod}(S).$$

These formulae are exact – not approximations – because $\text{req}(E_i)$ already incorporates every reaction internal to E_i , including those of every sub-ERC $E_j \subsetneq E_i$ (since $\mathcal{R}_{E_j} \subseteq \mathcal{R}_{E_i}$ by containment); the only cross-boundary reactions a combination can newly activate are exactly the ones captured by the fundamental-synergy hypergraph. S is self-maintaining iff $\text{req}(S) = \emptyset$; no separate SSM check or closure computation is ever needed.

Lemma 2 (Synergy ignition is a species-subset test, not a membership test). *When ERC set S grows to $S \cup \{j\}$, a fundamental synergy $E_a + E_b \rightarrow E_k$ fires (i.e. k is implied) iff $E_a \subseteq \text{sp}(S \cup \{j\})$ and $E_b \subseteq \text{sp}(S \cup \{j\})$ – not iff $a, b \in S \cup \{j\}$. These differ whenever a bigger ERC is present that species-wise dominates a minimal reactant ERC without that reactant itself ever being an explicit member of S : e.g. after a vertical lift (Section 8.2) injects an ancestor E of some $E_i \in S$, every descendant d of E satisfies $E_d \subseteq E \subseteq \text{sp}(S)$ and so is ignited, even though $d \notin S$.*

Lemma 2 is not a stylistic nuance: an implementation that tests $a, b \in S$ instead of the species-subset condition silently produces states whose tracked (sp, req, prod) disagree with the true reaction-level closure of $\text{sp}(S)$ – we found and fixed exactly this defect during validation (Section 10), and it is the reason Algorithm 7 below seeds its propagation queue not only from the literally added indices but from their full hierarchy descendant sets.

Deduplication across the whole search is a single global set of visited sp bitmasks: two different ERC-index paths that reach the same species set are recognised as the same module the instant

the second one is popped, with no separate “signature” object needed beyond the species bitmask itself.

8.2 Two modes, three extension moves

The search proceeds in two modes, mirroring the productive-novelty distinction of the companion theory: an open requirement can only be closed by attaching a *producer*, whereas a module that is already self-maintaining can only be grown further by attaching a *consumer* of something it already makes, by lifting to a bigger ERC, or by synergy.

Mode 1 (requirement-driven). While $\text{req}(S) \neq \emptyset$, for each still-required species s , attach every $\pi \in \text{comp_by_species}[s]$ (a $\subseteq$ -minimal producer of s); and for each $E_i \in S$, for each fundamental-synergy partner j of i whose target’s production intersects $\text{req}(S)$, attach j . Mode 1 halts the branch the instant $\text{req}(S) = \emptyset$ (a candidate SSM) or no attachment is possible (a dead end).

Mode 2 (growth past self-maintenance). Given an already self-maintaining S , three moves seed further growth, each resumed through Mode 1 in case it reopens a requirement:

1. *Synergy*: for $E_i \in S$, attach a fundamental-synergy partner $j \notin S$.
2. *Vertical lift*: for $E_i \in S$, attach a direct hierarchy parent E of E_i ($E \supsetneq E_i$). Since $\text{req}(E)$ already incorporates E_i ’s reactions, this is exactly another call to Algorithm 7 – no separate “absorb” step is needed.
3. *Consumer complementarity*: for each species s already produced by S ($s \in \text{prod}(S)$), attach a $\subseteq$ -minimal consumer $\gamma \in \text{comp_consumers_by_species}[s]$, $\gamma \notin S$. This is the mirror image of Mode 1’s producer attachment, using the same fundamental-complementarity relation from its other side.

Proposition 2 (Producer factorisation [1]). *Every requirement-closing attachment $S \rightarrow S \cup \{i\}$ with $s \in \text{prod}(E_i) \cap \text{req}(S)$ factors as a fundamental horizontal core followed by vertical lifts: producers of s form an up-set, so E_i dominates some $\hat{E} \in \text{minprod}(s)$; attaching $\hat{E}$ is a fundamental complementary extension, and E_i is recovered by climbing $\hat{E} \rightarrow \dots \rightarrow E_i$. When $\hat{E}$ is already an ancestor of a constituent, the horizontal step is vacuous and the attachment is purely vertical.*

This is exactly why Mode 1 restricts producer attachment to $\subseteq$ -minimal producers: any larger, non-minimal producer that could also close the requirement is reachable afterwards, if at all needed, by vertical lift from the minimal one – confining the branching factor to the fundamental relations rather than to $|\mathcal{E}|$ or $\binom{|\mathcal{E}|}{k}$.

Remark 2 (Canonical ordering). A DFS seeded from every non-persistent ERC would rediscover every multi-ERC SSM once per permutation of its constituents. Each seed state carries a `min_ext` gate: an explicit attachment must have index $\geq \text{min_ext}$, and any synergy-implied attachment with index below the seed’s own smallest constituent prunes the branch (its canonical path is from that smaller seed instead). This confines every $k!$ -fold permutation redundancy to a single canonical discovery, at the cost of no completeness: every SSM is still found, from its smallest-index constituent.

Algorithm 8 ComputeAllPersistentModules($\mathcal{E}$, hier, $\mathcal{F}_{\text{syn}}$, $\mathcal{F}_{\text{comp}}$)

```
1: // Single-ERC EPMs (no search needed)
2:  $\text{EPM}_1 \leftarrow \{E_i : \text{req}(E_i) = \emptyset, \nexists E_j \subsetneq E_i, \text{req}(E_j) = \emptyset\}$ 
3: // Mode 1: one canonical-order seed per non-persistent ERC
4: for each non-persistent  $E_i$  do
5:   seed a DFS state  $\{i\}$  with  $\text{min\_ext} = i+1$   $\triangleright$  only extensions  $\geq i+1$  allowed – see Remark 2
6: end for
7: run Mode 1 from every seed until every branch is an SSM or a dead end; collect  $\text{SSM}_1$ 
8: // Order assignment (post-hoc, by species-mask containment)
9: sort  $\text{EPM}_1 \cup \text{SSM}_1$  by popcount; assign  $\text{ord}(X) = 1 + \max\{\text{ord}(Y) : Y \subsetneq X, Y \text{ already ordered}\}$ ,
   else 0
10: // Mode 2: BFS by order
11: for  $k = 1, 2, \dots$  until no new module do
12:   for each  $X_p$  with  $\text{ord}(X_p) = k - 1$  do
13:     seed Mode 2 extensions (synergy, vertical lift, consumer complementarity)
14:     resume Mode 1 from each seed; new SSMs get order via the same containment rule
15:   end for
16: end for
17: return all discovered SOs, grouped by order
```

8.3 The generated core, and latent joins on demand

Definition 17 (Latent join). $X \in \text{SO}$ is latent if some irreducible generator of X consists entirely of steps that neither fire a new reaction nor strictly reduce a requirement – equivalently, X is the join of strictly smaller SOs sharing no synergy or complementarity between them. Write SO^{gen} for the non-latent (generated) modules.

Proposition 3 (Latent-generated factorisation [1]). $\text{SO} = \langle \text{SO}^{\text{gen}} \rangle_{\vee}$: every SO is a join of generated modules, and because $\text{req}(X \vee Y) \subseteq \text{req}(X) \cup \text{req}(Y)$ always, joining two SSM modules is automatically SSM – so nothing about a latent join needs to be searched for; it is a single closure computation away from any two of its generated pieces.

Algorithm 8 therefore never attaches an ERC purely because it happens to share species with the current module and produces no productive novelty: Mode 1 only attaches producers of an *open* requirement, and Mode 2’s three moves are all instances of productive novelty (synergy, complementarity, or absorbing a descendant already present). The result is that Algorithm 8 computes exactly SO^{gen} , not SO – which is the escape from the combinatorial explosion, since the latent joins of otherwise-unrelated modules already number exponentially in a network with several independent persistent components. A latent module is reconstructed on demand in $O(1)$ closure computations rather than ever being searched for:

Complexity. Each Mode-1/Mode-2 state transition is $O(1)$ amortised bitmask work per ERC touched by Algorithm 7 (no reaction scan). Branching per open state is bounded by the number of fundamental synergy partners and $\subseteq$ -minimal producers/consumers touching the state – the fundamental-relation density – never by $|\mathcal{E}|$ or $\binom{|\mathcal{E}|}{2}$; the companion study’s empirical finding that this density *shrinks* as networks grow [1] is what makes the search tractable at genome scale (Section 11). The one remaining global-complexity step is order assignment (line 8 of Algorithm 8),

Algorithm 9 LatentJoin(X, Y)

```
1:  $Z \leftarrow \text{cl}(X \mid Y)$ 
2: if  $\text{req}(Z) = \emptyset$  and  $Z$  is connected then
3:   return  $Z$ 
4: else
5:   return NONE
6: end if
```

a containment sweep over all discovered SOs found so far: $O(|SO^{\text{gen}}|^2)$ in the worst case, the dominant cost on networks with thousands of ESPMs (Section 11).

9 Bitset Implementation

All species and ERC sets are represented as Python `int` values used as n -bit integers (species $s_i \leftrightarrow \text{bit } i$), giving union, intersection, containment, and complement as single machine bitwise operations; Python integers are arbitrary precision, so networks with more than 64 species are handled transparently.

- **ERCData** (frozen, one per ERC): `species_mask`, `req_mask`, `prod_mask`, `min_bases` (list of bitsets), `reaction_indices`.
- **HierarchyData**: per-ERC `parents` (direct Hasse covers), `children`, `ancestors`, `descendants` (transitively closed frozensets).
- **FundamentalGraph**: precomputed once per network from the three structures above – `syn_from[i] = \{(j, k) : E_i + E_j \rightarrow E_k\}` (stored symmetrically in i and j), `comp_by_species[s] = minprod(s)`, `comp_consumers_by_species[s] = mincons(s)`.
- **DFSState** (immutable): (`erc_set`, `sp`, `req`, `prod`, `min_ext`) – everything Algorithm 7 needs, and everything the search visits or dedups on.

The whole EPM/ESPM search touches the reaction network only twice: once to build $\mathcal{E}$ (Section 3) and once, for each ERC, to precompute req/prod from $\mathcal{R}_E$. Every subsequent state transition is pure ERC-index-space bitmask arithmetic.

10 Oracle-First Validation

Each stage has an independent brute-force oracle used exclusively for cross-validation, never in the production pipeline:

- **erc_oracle**: enumerates all $X \subseteq M$ and checks closure directly – exponential in $|M|$, usable for small hand-built networks.
- **synergy_oracle**: a plain triple loop, filtered basic $\rightarrow$ maximal $\rightarrow$ fundamental exactly per Definitions 10 (used to validate both Algorithm 2 and Algorithm 3).
- **comp_basic_set** / **comp_fund.set**: plain pair and species loops computing basic complementarity and `minprod(s)/mincons(s)` directly from Definitions 3–5.

- `epm_oracle` / `espm_oracle`: enumerate all $2^{|\mathcal{E}|}$ ERC subsets, compute the joint closure of each, and keep those that are closed, SSM, reactive, and connected; the $\subseteq$ -minimal such sets are the EPMs, and order is assigned by the same containment rule as Definition 6. Feasible up to $|\mathcal{E}| \lesssim 24$.

10.1 Validation methodology and results

The EPM/ESPM search was validated by exact-set comparison against `epm_oracle`/`espm_oracle` on (i) five hand-built networks covering loops, hierarchies, non-persistence, and inflow quotienting; (ii) a fully worked network (Section 10.2) exhibiting every mechanism, including a sterile ERC; and (iii) 25 real BioModels networks with 4–24 ERCs, the size at which the oracle itself remains tractable. Because Algorithm 8 deliberately searches only SO^{gen} (Section 8), agreement with the oracle – which enumerates all of SO – is defined modulo latent joins: every module found is required to be a genuine member of SO (soundness), and every oracle module absent from the result is required to be reconstructible by repeatedly applying Algorithm 9 to modules already found (completeness of the generated core). Both conditions held exactly on all 30 test networks. This process caught two real defects during development, both now fixed and covered by regression tests: (a) an earlier version of Mode 2 seeded growth past an SSM only through synergy edges, so it silently returned zero ESPMs on any network whose novelty was purely complementarity- or lift-driven – exactly the worked example of Section 10.2; and (b) fixing (a) exposed Lemma 2’s membership-vs-subset distinction, an earlier version of Algorithm 7 tested ERC-index membership rather than species-subset containment and briefly produced states whose tracked masks disagreed with their true reaction-level closure on a real BioModels network before the fix.

10.2 A fully worked network

Let $M = \{a, b, c, d, e, f, g, h, k, p, q, w, z\}$ and

$$\begin{aligned} r_A &= f+a \rightarrow 2a+p, & r_B &= p+b \rightarrow 2b+f, & r_C &= p+c \rightarrow 2c+f, \\ r_D &= g+d \rightarrow 2d+q, & r_E &= q+e \rightarrow 2e+g, & r_F &= p+h \rightarrow 2h, \\ r_H &= p+z \rightarrow z+b, & r_G &= k+w \rightarrow 2k+p. \end{aligned}$$

This gives eight ERCs (Table 1); the only containment is $E' \supset E_B$ ($E' = \text{cl}\{p, z\} = \{p, z, b, f\}$, since r_H makes b , which ignites r_B). There are no synergies – every reaction ignites from a single ERC’s closure – so every relation here is complementary, the cleanest setting to exercise Mode 2.

Algorithm 8 finds exactly the three EPMs $Y_1 = E_A \vee E_B = \{f, a, p, b\}$, $Y_2 = E_A \vee E_C = \{f, a, p, c\}$, and (a disjoint component) $Y_3 = E_D \vee E_E = \{g, d, q, e\}$, and exactly the following eight ESPMs, matching `espm_oracle` exactly:

- order 1: $\{f, a, p, b, c\}$ ($= Y_1 \vee Y_2$, a latent join),
 $\{f, a, p, b, h\}$ ($= Y_1 \vee E^*$, consumer complementarity),
 $\{f, a, p, c, h\}$ ($= Y_2 \vee E^*$),
 $\{f, a, p, b, z\}$ ($= Y_1 \vee E'$, vertical lift over E_B);
- order 2: $\{f, a, p, b, c, h\}$, $\{f, a, p, b, c, z\}$, $\{f, a, p, b, h, z\}$;
- order 3: $\{f, a, p, b, c, h, z\}$.

$Y_1 \vee Y_2$ (order 1) is discovered as a plain consumer-complementarity attachment: E_C is a fundamental consumer of p , which Y_1 already produces via E_A , so it is found by exactly the same mechanism

ERC	species	req
$E_A = \text{cl}\{f, a\}$	$\{f, a, p\}$	$\{f\}$
$E_B = \text{cl}\{p, b\}$	$\{p, b, f\}$	$\{p\}$
$E_C = \text{cl}\{p, c\}$	$\{p, c, f\}$	$\{p\}$
$E_D = \text{cl}\{g, d\}$	$\{g, d, q\}$	$\{g\}$
$E_E = \text{cl}\{q, e\}$	$\{q, e, g\}$	$\{q\}$
$E^* = \text{cl}\{p, h\}$	$\{p, h\}$	$\{p\}$
$E' = \text{cl}\{p, z\}$	$\{p, z, b, f\}$	$\{p\}$
$E_{\dagger} = \text{cl}\{k, w\}$	$\{k, w, p\}$	$\{w\}$

Table 1: ERCs of the worked network. $E_{\dagger}$ produces p but requires w , which no reaction produces: Mode 1 seeded from $E_{\dagger}$ dead-ends the moment w has no complementary producer, so $E_{\dagger}$ ’s species never appear in any discovered module – with no separate sterility fixpoint required to establish this in advance.

as the genuine discoveries – Definition 9’s “latent” classification is a statement about *whether* a step was productively novel along some irreducible generator, not about which search mechanism reaches a module, and $Y_1 \vee Y_2$ is simultaneously reachable as a latent join and as a consumer-complementarity attachment; excluding it from the search would require tracking generator irreducibility explicitly, which the current implementation does not do (Section 12). $\{f, a, p, b, z\}$ is the module that specifically requires the vertical-lift move (item 2 of Section 8.2): E' ’s minimal producer of p is E_A , already present, but E' itself is not a *minimal* consumer of p (its sub-ERC E_B is, and is already present too), so Algorithm 5 never registers E' as a fundamental complementarity partner of anything – the only route to it is lifting directly from its hierarchy parent relation to E_B .

11 Experiments

We report one representative genome-scale run rather than a full-corpus sweep (left for future work, Section 12). On the BiGG reconstruction iNJ661 (825 species, 1296 reactions), the pipeline finds 712 ERCs (210 persistent) in 3.7 s; the containment hierarchy in 0.06 s; 3,697 fundamental synergies (of 240,833 basic, 123,451 maximal) via Algorithm 3 in 2.9 s; 1,097 fundamental complementarities (of 6,738 basic, 3,139 pure) in 0.2 s; 329 primitive ERCs reaching 87.9% synergy coverage of $\mathcal{E}$ in 1 ms; and Algorithm 8 finds all 72 EPs and 1,124 ESPs across 17 orders in 12.5 s (17,997 states explored: 1,124 SSs, 8,271 dead ends, the remainder branching states), for a total pipeline time of 19.4 s on a single desktop core – a regime in which the 2^{712} -subset brute-force oracle is entirely infeasible.

12 Conclusion

We have described, implemented, and validated a complete pipeline for the generative structure of chemical organizations: ERC discovery, the containment hierarchy, basic/maximal/fundamental synergy (with a basis-first algorithm markedly faster than pair-first on sparse networks), basic/pure/fundamental complementarity, primitive-ERC synergy reachability, and – the paper’s central contribution – exact computation of every EPM and every ESPM of any order, via a two-mode ERC-index-space traversal that never recomputes a reaction-level closure and deliberately confines its search to the generated core, excluding latent joins in favour of an $O(1)$ on-demand reconstruc-

tion. This closes the gap left open by the companion existence theory [1] and by an earlier version of this pipeline that covered only single-ERC and pairwise EPMs; correctness is established not by proof sketch alone but by exhaustive oracle cross-validation on hand-built, worked, and real networks.

Concrete extensions this implementation makes natural, and which we leave as future work: (i) precomputing the requirement-reachability fixpoint that identifies *sterile* ERCs – those whose requirement can never be reduced to $\emptyset$ by any reachable combination – so that provably dead branches are pruned before rather than after being explored, as sketched (but not yet implemented) in the companion algorithmic theory; (ii) an explicit generator-irreducibility check so that latent joins reachable through consumer complementarity (as in Section 10.2) are recognised and excluded from the search rather than incidentally found by it; (iii) grade tracking – the lowest order at which an ERC first becomes load-bearing – as a structural, per-module summary statistic; (iv) replacing the $O(|SO^{\text{gen}}|^2)$ post-hoc order-assignment sweep with an incremental, order-indexed dedup structure for networks with tens of thousands of ESPMs; and (v) a full-corpus benchmark, in the style of the companion study’s 438-network survey, now measuring computation rather than counts.

References

- [1] T. Veloz and A. Bassi, “Synergy and Complementarity: The Generative Basis of Chemical Organizations,” *Bulletin of Mathematical Biology*, in press.
- [2] P. Dittrich and P. Speroni di Fenizio, “Chemical Organisation Theory,” *Bulletin of Mathematical Biology*, vol. 69, no. 4, pp. 1199–1231, 2007.
- [3] W. F. Dowling and J. H. Gallier, “Linear-time algorithms for testing the satisfiability of propositional Horn formulae,” *Journal of Logic Programming*, vol. 1, no. 3, pp. 267–284, 1984.
- [4] Z. King *et al.*, “BiGG Models: A platform for integrating, standardizing, and sharing genome-scale models,” *Nucleic Acids Research*, vol. 44, no. D1, pp. D515–D522, 2016.